

Perancangan Knight-Chaos Cipher: Algoritma Kriptografi Berbasis Langkah Kuda Catur dan Chaotic Map

Nama: Muhammad Rizky Hajar

NIM: 24.55.2714

Program Studi: PJJ Informatika

Konsentrasi: Big Data dan Predictive Analytics (BDPA)

Mata Kuliah: Cyber Security

1. Pendahuluan

Kriptografi digunakan untuk menjaga kerahasiaan, integritas, dan autentikasi data. Algoritma cipher modern umumnya bekerja dengan prinsip substitution dan permutation yang diulang beberapa kali (round) untuk mengubah plaintext menjadi ciphertext. Pada tugas ini, saya merancang algoritma kriptografi baru bernama **Knight-Chaos Cipher (KCC-128)**.

Ide utama KCC-128 adalah memodifikasi komponen inti cipher dengan cara yang berbeda dari biasanya. Alih-alih menggunakan S-Box tetap seperti AES, saya menggunakan **logistic chaotic map** untuk men-generate S-Box yang berubah setiap round. Untuk bagian permutation, alih-alih ShiftRows yang linear, saya menggunakan pola **langkah kuda catur** pada board 4×4 sebagai mekanisme shuffling posisi byte.

2. Konsep Dasar Algoritma

KCC-128 bekerja sebagai block cipher dengan block size 128 bit (16 byte). Setiap block diperlakukan sebagai matrix 4×4. Algoritma berjalan selama 10 round, dimana setiap round menggunakan key material yang berbeda: round key, dynamic S-Box, inverse S-Box, knight permutation path, inverse permutation, dan rotation pattern.

Key dari user diproses dengan SHA-256 untuk menghasilkan master seed. Dari master seed ini, semua round material di-derive. Yang menarik adalah key schedule tidak hanya menghasilkan round key untuk XOR, tapi juga menentukan bentuk S-Box, route permutation, dan jumlah bit rotation. Jadi, key benar-benar mengubah seluruh struktur transformasi — bukan hanya nilai XOR-nya.

3. Arsitektur Knight-Chaos Cipher

3.1 Dynamic S-Box Berbasis Chaotic Map

S-Box pada KCC-128 tidak tetap. Setiap round punya S-Box sendiri yang di-generate menggunakan logistic chaotic map:

$$x(n+1) = r * x(n) * (1 - x(n))$$

Initial value x dan parameter r di-derive dari round seed. Deret chaotic ini dipakai untuk memberi score pada setiap byte value 0–255, kemudian byte di-sort berdasarkan score tersebut.

Hasilnya adalah permutation 0..255 yang berfungsi sebagai S-Box. Karena S-Box berupa permutation (bijective), inverse-nya bisa langsung dihitung untuk decryption.

3.2 Permutasi Langkah Kuda Catur

Setiap block 16 byte di-map ke board 4×4. Posisi byte lalu di-permute mengikuti pola langkah kuda catur (L-shape: 2+1 cell). Starting position ditentukan oleh round seed. Kalau ada lebih dari satu valid move, dipilih yang punya jumlah onward move terkecil (Warnsdorff's heuristic). Kalau masih tie, round seed dipakai sebagai tiebreaker.

Dibanding shift linear seperti ShiftRows di AES, knight move menghasilkan perpindahan yang non-sequential antar row dan column. Byte yang tadinya adjacent bisa ter-scatter jauh setelah permutation.

3.3 BitShiftMix dan Byte Diffusion

Setelah substitution dan permutation, setiap byte di-rotate left sejumlah bit (jumlahnya ditentukan round key). Lalu dilakukan byte diffusion: setiap byte di-XOR dengan byte sebelumnya secara chaining. Terakhir ada Feistel-like mixing yang mencampur byte secara pair-wise dalam 3 pass.

Kenapa perlu layer ini? Substitution mengacak value byte, permutation mengacak posisi byte, tapi keduanya belum menyebarkan perubahan antar byte. Diffusion dan Feistel mixing memastikan kalau satu byte berubah, efeknya propagate ke byte-byte lain.

3.4 Mode Chaining (CBC)

Untuk message yang lebih panjang dari 16 byte, KCC-128 menggunakan mode CBC (Cipher Block Chaining). Block pertama di-XOR dengan IV yang di-derive dari key, block kedua dan seterusnya di-XOR dengan ciphertext block sebelumnya. Tujuannya agar block plaintext yang sama di posisi berbeda tidak menghasilkan ciphertext yang sama.

4. Proses Encryption dan Decryption

Pada encryption, plaintext diberi PKCS#7 padding supaya panjangnya menjadi multiple of 16 byte. Setiap block melewati 10 round dengan urutan:

1. **AddRoundKey** — state di-XOR dengan round key
2. **ChaoticSubBytes** — setiap byte di-substitute pakai dynamic S-Box
3. **KnightPermutation** — posisi byte di-shuffle pakai knight path
4. **BitShiftMix** — bit di dalam setiap byte di-rotate
5. **ByteDiffusion** — XOR chaining antar byte
6. **FeistelMix** — pair-wise mixing 3 pass

Pada decryption, semua step di-reverse dan menggunakan inverse operation. S-Box punya inverse S-Box, permutation punya inverse permutation, rotate left dibalik dengan rotate right, dan seterusnya. Karena semua operation dirancang invertible, plaintext bisa di-recover dengan tepat.

Komponen	Fungsi	Keunikan
AddRoundKey	XOR state dengan round key	Round key dari SHA-256
ChaoticSubBytes	Substitute value byte	S-Box berubah tiap round (chaotic map)
KnightPermutation	Shuffle posisi byte	Knight's tour pattern pada board 4×4
BitShiftMix	Rotate bit dalam byte	Rotation amount dari round seed
ByteDiffusion	Propagate perubahan antar byte	XOR chaining dengan carry
FeistelMix	Strengthen diffusion	Invertible mixing 3 pass

Pseudocode encryption satu block:

```

state <- block
FOR each round DO
    state <- state XOR roundKey
    state <- SBox[state]
    state <- KnightPermutation(state)
    state <- RotateLeft(state)
    state <- ByteDiffusion(state)
    state <- FeistelMix(state)
END FOR
RETURN state

```

5. Metodologi Pengujian

Pengujian dilakukan untuk tiga hal:

1. **Correctness** — apakah `decrypt(encrypt(plaintext)) == plaintext`?
2. **Entropy** — apakah byte distribution ciphertext lebih uniform dibanding plaintext?
3. **Avalanche effect** — kalau 1 bit plaintext diubah, berapa persen bit ciphertext yang ikut berubah?

Rumus Shannon entropy:

$$H(X) = - \sum p(x) \log_2 p(x)$$

Maximum entropy adalah 8 bit/byte (perfectly uniform distribution). Untuk avalanche effect, idealnya sekitar 50% — artinya perubahan kecil pada input mengubah kira-kira setengah bit output.

Selain demo CLI (Python), dibuat juga demo interaktif berbasis web yang menampilkan matrix 4×4 dan perubahan state per step.

6. Hasil Implementasi dan Sample Run

Implementasi menggunakan Python. Hasil sample run:

- **Decryption OK:** True (plaintext kembali utuh)
- **Plaintext entropy:** ~4.38 bit/byte
- **Ciphertext entropy:** ~6.63 bit/byte
- **Avalanche effect:** ~48.34%

Entropy naik dari 4.38 ke 6.63, menunjukkan ciphertext punya byte distribution yang lebih uniform dibanding plaintext (yang berupa teks biasa dengan karakter berulang). Avalanche 48.34% artinya mengubah 1 bit pada plaintext menyebabkan hampir separuh bit ciphertext berubah.

Demo interaktif web juga berjalan. User bisa memasukkan plaintext dan key, menekan tombol Encrypt, lalu melihat perubahan state round per round.

7. Analisis Keamanan

Entropy: Ciphertext menunjukkan byte distribution yang lebih uniform dibanding plaintext. Pattern karakter yang berulang pada teks natural sudah tidak terlihat pada ciphertext.

Avalanche effect: Dengan mengubah 1 bit pertama pada plaintext, sekitar 48% bit ciphertext berubah. Ini menunjukkan bahwa kombinasi dynamic S-Box, knight permutation, rotation, diffusion, dan Feistel mixing bekerja cukup baik dalam men-propagate perubahan.

Frequency analysis: Byte yang paling sering muncul pada ciphertext punya frekuensi rendah dan tersebar — tidak ada dominasi satu value tertentu.

Perbandingan dengan cipher klasik: Caesar dan Vigenere rentan terhadap frequency analysis karena hubungan plaintext–ciphertext masih langsung (substitution per karakter/posisi). KCC-128 memutus hubungan ini melalui non-linear S-Box, permutation yang men-shuffle posisi, dan diffusion yang men-propagate perubahan antar byte.

Pengujian lanjutan yang bisa dilakukan: differential cryptanalysis, linear cryptanalysis, randomness test dengan dataset besar, dan evaluasi terhadap chosen-plaintext attack.

8. Kelebihan dan Keterbatasan

Kelebihan: - S-Box tidak tetap, berubah setiap round berdasarkan chaotic map - Non-linear permutation dari knight's tour pattern - Key schedule memengaruhi semua komponen (bukan hanya round key) - Semua operation invertible sehingga decryption bekerja dengan benar

Keterbatasan: - IV di-derive secara deterministic dari key (untuk konsistensi demo). Idealnya IV random dan disimpan bersama ciphertext - Pengujian masih menggunakan sample kecil. Test dengan data lebih besar bisa memberi gambaran yang lebih akurat - Belum ada mekanisme authentication (MAC/HMAC). Cipher hanya menjamin confidentiality, belum integrity - Kualitas kriptografis S-Box (nonlinearity, differential uniformity) belum diuji secara formal

9. Kesimpulan

Knight-Chaos Cipher (KCC-128) berhasil dirancang dan diimplementasikan sebagai block cipher yang memodifikasi komponen substitution, permutation, diffusion, dan key schedule.

Knight's tour dipakai sebagai mekanisme permutation, dan logistic chaotic map dipakai untuk men-generate dynamic S-Box per round.

Dari pengujian awal, encryption-decryption berjalan benar, ciphertext entropy meningkat signifikan, dan avalanche effect mendekati 50%.

Referensi

1. William Stallings, *Cryptography and Network Security: Principles and Practice*.
2. Joan Daemen dan Vincent Rijmen, *The Design of Rijndael: AES - The Advanced Encryption Standard*.
3. Bruce Schneier, *Applied Cryptography*.
4. Robert L. Devaney, *An Introduction to Chaotic Dynamical Systems*.